

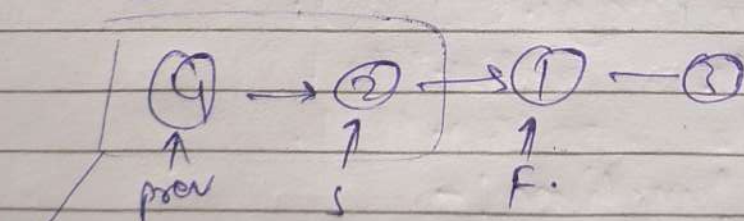
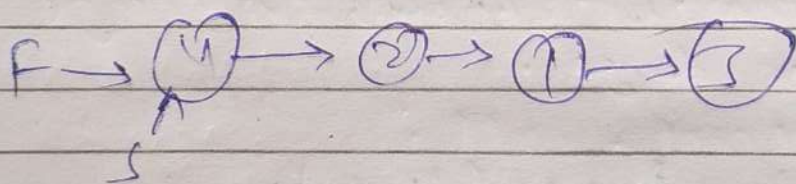
Q13 head = ④ → ② → ① → ③

hme isha list ko sort karna hai jo ki hm karenge :-

- ① phle mid find karenge.
- ② usko do part me split karenge.
- ③ Merge sort ki help se sort karenge.

→ By Slow-Fast approach hm list me mid node tak pohuchenge.

kyunki slow ek step aage jata hai aur fast 2 step aage jata hai.



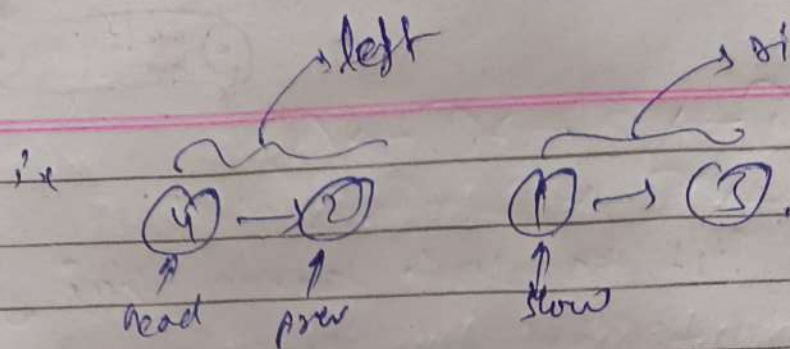
→ split karo.

ie ek loop me karenge.

```
while (fast != null & fast.next != null) {  
    prev = slow; // slow ke piche slow रहेगा.  
    slow = slow.next // slow ko 1 step aage.  
    fast = fast.next.next // fast ko 2 step aage.  
}
```

prev.next = null;

→ Ise list 2 parts me divide kardega.



Ab dono ko sort karenge.

ListNode L1 = sortList(head);
 Use left wala part sort hoga.

ListNode L2 = sortList(slow);
 Use right wala part sort hoga.

→ Phir hm dono list (L1, L2) ko merge karke return kar denge.

return merge(L1, L2);

→ Ab merge function:-

ek dummy node create karenge aur usme
 0 store kar denge.

→ Ab ~~curr~~ dummy ko curr me store kar denge
 curr pointer list banana ke liye.

ListNode curr = dummy;

→ Ab Merge Loop lagayenge.

while (a != null && b != null) // jab tak dono list me element hai,

if (a.val <= b.val) {

curr.next = a;

a = a.next;

} else {

curr.next = b;

b = b.next;

}

curr = curr.next // curr ko aage badayenge.

→ Remaining Nodes // agr koi node bach gata hai end me toh.

if (a != null) {

curr.next = a;

}

if (b != null) {

curr.next = b;

}

→ Aur at the end result return krna hai

return dummy.next;

// Code 5

class Solution

public static void sortList(ListNode head) {

if (head == null || head.next == null) {
 return head;
}

ListNode prev = null;

ListNode slow = head;

ListNode fast = head;

while (fast != null && fast.next != null) {

prev = slow;

slow = slow.next;

fast = fast.next.next;

}

prev.next = null;

ListNode l1 = sortList(l1);

ListNode l2 = sortList(l2);

return merge(l1, l2);

}

private static void mergeSort(ListNode a, ListNode b) {

while (a != null && b != null) {

if (a.val < b.val) {

~~curr = a;~~

ListNode dummy = new ListNode(0);

ListNode curr = dummy;

while (a != null && b != null) {

if (a.val < b.val) {

curr.next = a ;

a = a.next ;

} else {

curr.next = b ;

b = b.next ;

}

curr = curr.next ;

}

if (a != null) {

curr.next = a ;

}

if (b != null) {

curr.next = b ;

}

return dummy.next ;

}

}